

Quantum Convolutional Neural Networks for Image classification

Miguel Curto Castela, miguelcastela@student.dei.uc.pt

Avisor: Professor Jorge Lobo, jlobo@deec.uc.pt

work in the scope of the UC course: Quantum Algorithms and Applications ; Specialised training Course in Quantum Computing and Technologies.

Abstract

Neural networks are currently among the most widely used models in machine learning. A sub-category of these, known as convolutional neural networks, is primarily applied to image recognition tasks. By using pooling and convolution layers, CNNs can extract features from images, making them effective for image classification. They have been showing promising results when compared to classical methods. This project explores the integration of quantum computing with neural network architectures, focusing on two types of quantum neural networks (QNNs) from the Qiskit tutorials. A final neural network was then created with the best features of each of the tutorials and some evolutionary-inspired approaches to the hybrid quantum neural network (HQCNN)

Introduction

First tutorial

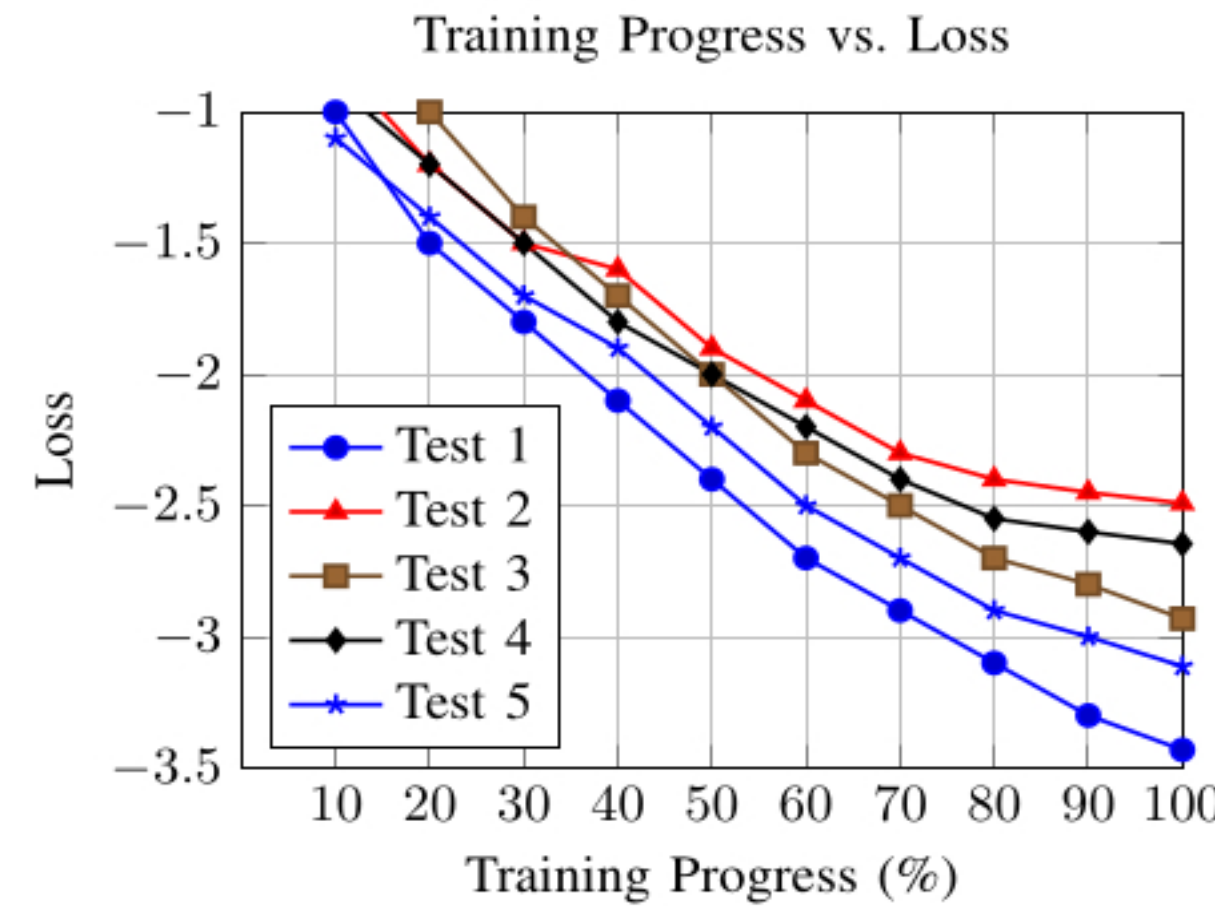
The first tutorial focuses on a more direct application of a HQCNN using a 2-qubit quantum layer, optimized with the Adam optimizer. It begins with classical processing (conv2D and pooling). The output of the classical layers is then passed to a quantum circuit that uses a ZZFeatureMap [1] and a RealAmplitudes ansatz with TorchConnector [2]. The output is passed back to a final classical layer for prediction.

Second tutorial

In contrast, the second tutorial adopts a fully quantum model with custom built quantum convolutional and pooling layers. It uses the ZfeatureMap [3] on 8 qubits and is trained using COBYLA, a no gradient optimizer. The output is measured with a sparse Pauli observable on the final state

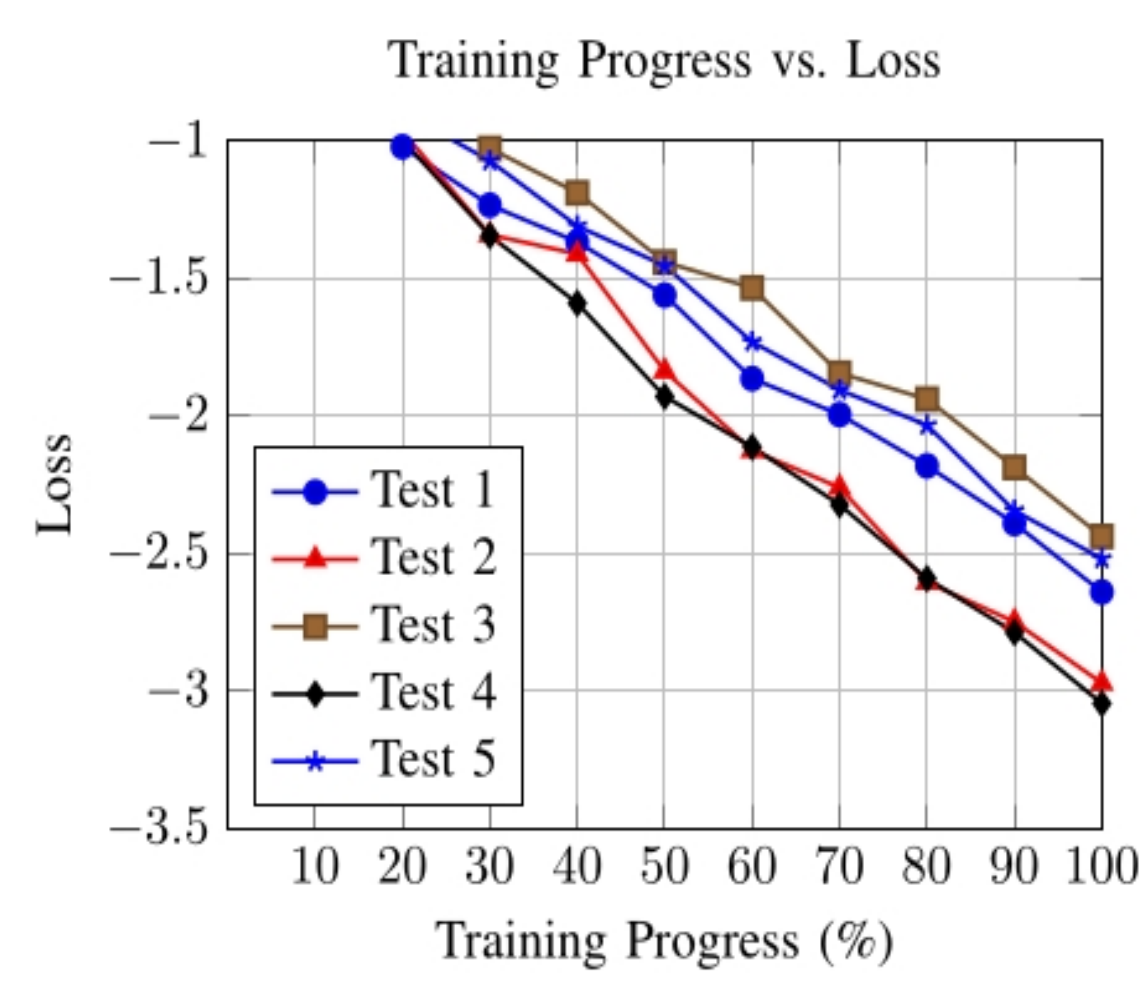


	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-3.4312	-2.4912	-2.9313	-2.6458	-3.1129
accuracy	100%	99%	100%	100%	100%

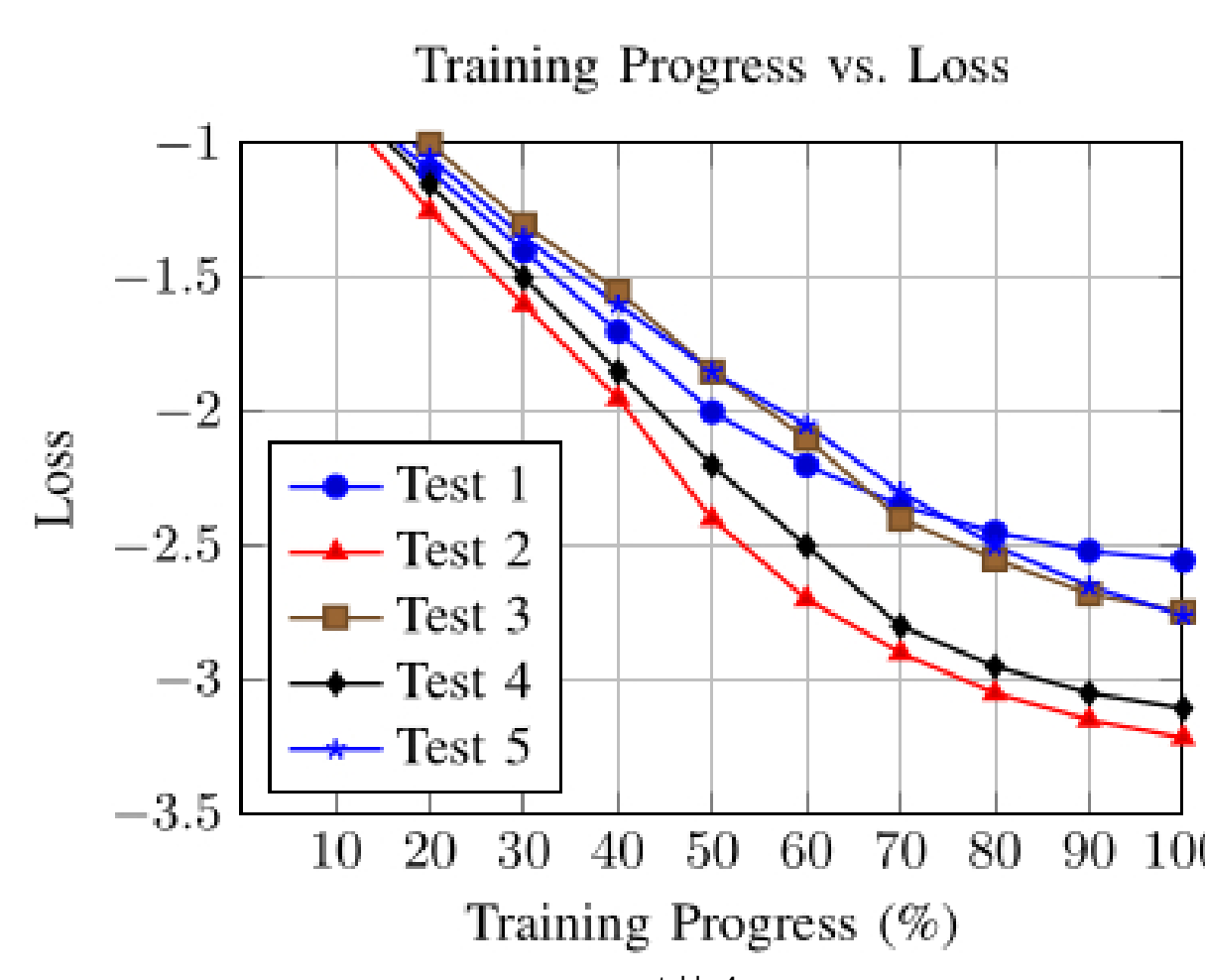


	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-3.5102	-2.5000	-2.9971	-2.6358	-3.4921
accuracy	100%	99%	100%	100%	100%

Results of 5 tests using the MNIST hand-written digits dataset [4] for binary classification (50 test samples and 50 training samples, for 10 epochs) for the first tutorial (table 1) and second tutorial (table2) for 200 iterations using COBYLA.



	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-2.5541	-3.2172	-2.7470	-3.1065	-2.7612
accuracy	99%	100%	100%	100%	100%



	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-2.5012	-3.2629	-2.7570	-3.1023	-2.5610
Accuracy	43%	75%	63%	62%	54%

Results of 5 tests using a synthetic dataset of vertical and horizontal lines (50 test samples and 50 training samples, for 10 epochs/200 iterations when using COBYLA) for the first tutorial (table 3) and second tutorial (table 4).

References

- [1] "Zzfeaturemap class – qiskit documentation," ZZFeatureMap, 2025. <https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.ZZFeatureMap>.
- [2] PyTorch Contributors, "torch.utils.data – pytorch 2.7 documentation, 2025. <https://pytorch.org/docs/stable/data.html>, 202.5
- [3] Qiskit Contributors, "Zfeaturemap class – qiskit documentation," library.ZFeatureMap, 2025. <https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.ZFeatureMap>.
- [4] Y. LeCun and C. Cortes, "MNIST handwritten digit database," 2010. <http://yann.lecun.com/exdb/mnist/>
- [5] Twolocal ansatz – qiskit documentation, 2025, <https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.TwoLocal>

Results

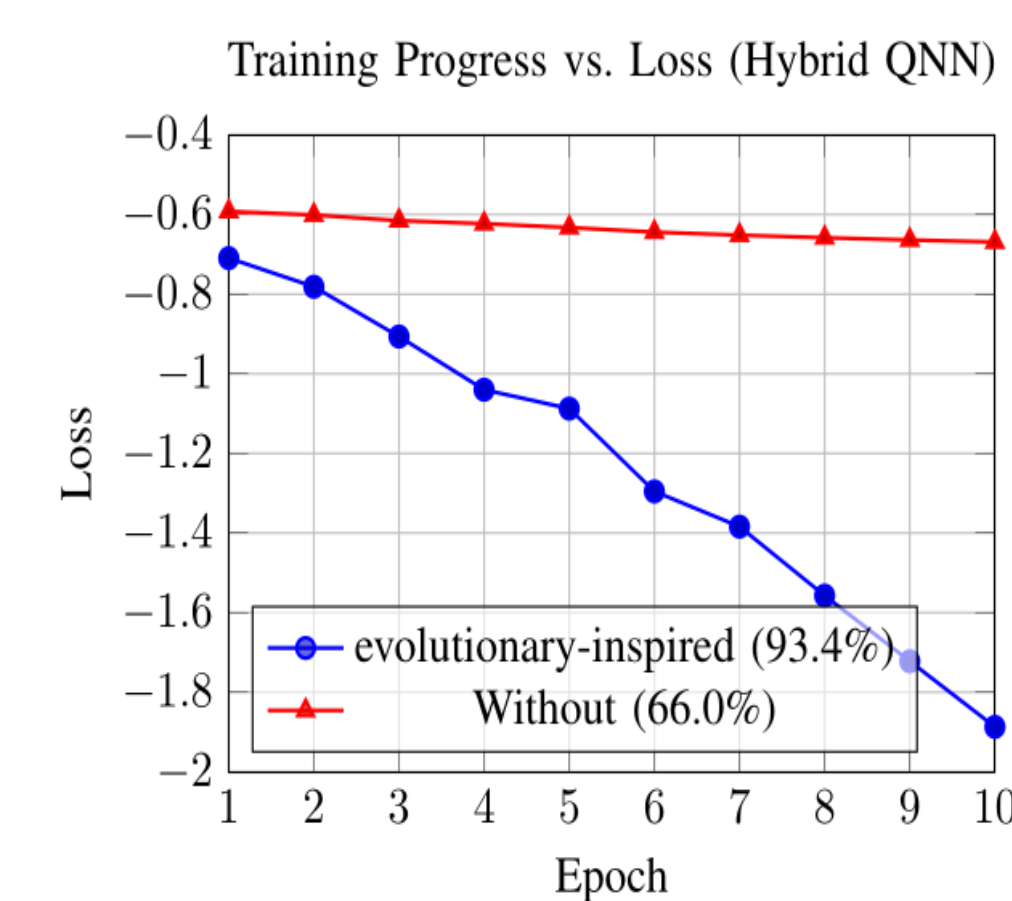
The second tutorial lacks any form of classical preprocessing or feature extraction, making it harder for the quantum model to generalize from high-dimensional input data. The usage of zfeature map (although with more qubits) also limits the model's ability to capture complex patterns compared to the first tutorial's classical CNN layers. Additionally, the second tutorial's use of a derivative-free optimizer (COBYLA) contrasts with the first tutorial's Adam optimizer, which is more suited for gradient-based learning in hybrid models. These factors contribute to the observed differences in performance and accuracy between the two tutorials.

Creating a custom HQCNN

Inspired by the best-performing features of the tutorials and additional ideas, a custom HQCNN was developed. It integrates classical pre-processing, quantum encoding, and evolutionary optimization techniques. Quantum image encoding methods such as FRQI and NEQR, which represent pixel intensity and position in quantum states, were initially explored. Ultimately, the ZZFeatureMap was selected due to its demonstrated ability to capture complex feature interactions.

The quantum circuit was constructed using a flexible and customizable *TwoLocal* ansatz [5] with *ry* rotations and *cz* entanglement gates. For optimization, the Adam optimizer was employed to fully leverage PyTorch's autograd system, thereby avoiding the limitations of COBYLA in hybrid setups. One quantum convolution and one pooling layer were implemented, drawing inspiration from the second tutorial. The output layer was defined using nn.Sequential, consisting of a linear layer followed by a softmax activation to facilitate straightforward binary classification.

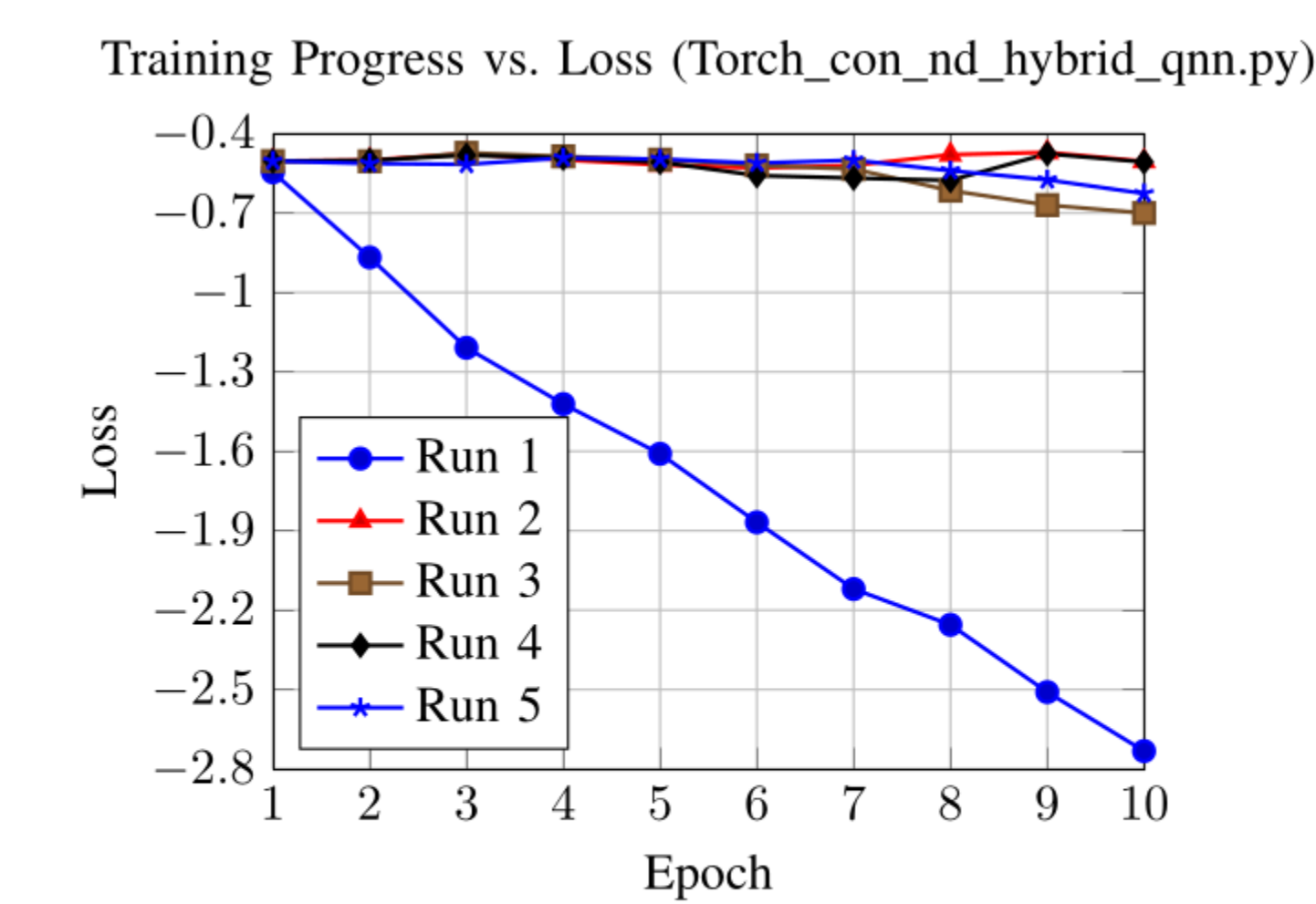
To further enhance model performance, evolutionary operators were incorporated into the tensors parameters. These included a single-point crossover and Gaussian mutation with a standard deviation applied at the end of each epoch. This introduced beneficial stochastic variability and contributed to improved convergence. As epochs progressed, increasing model stability in learning was observed as a result of these enhancements.



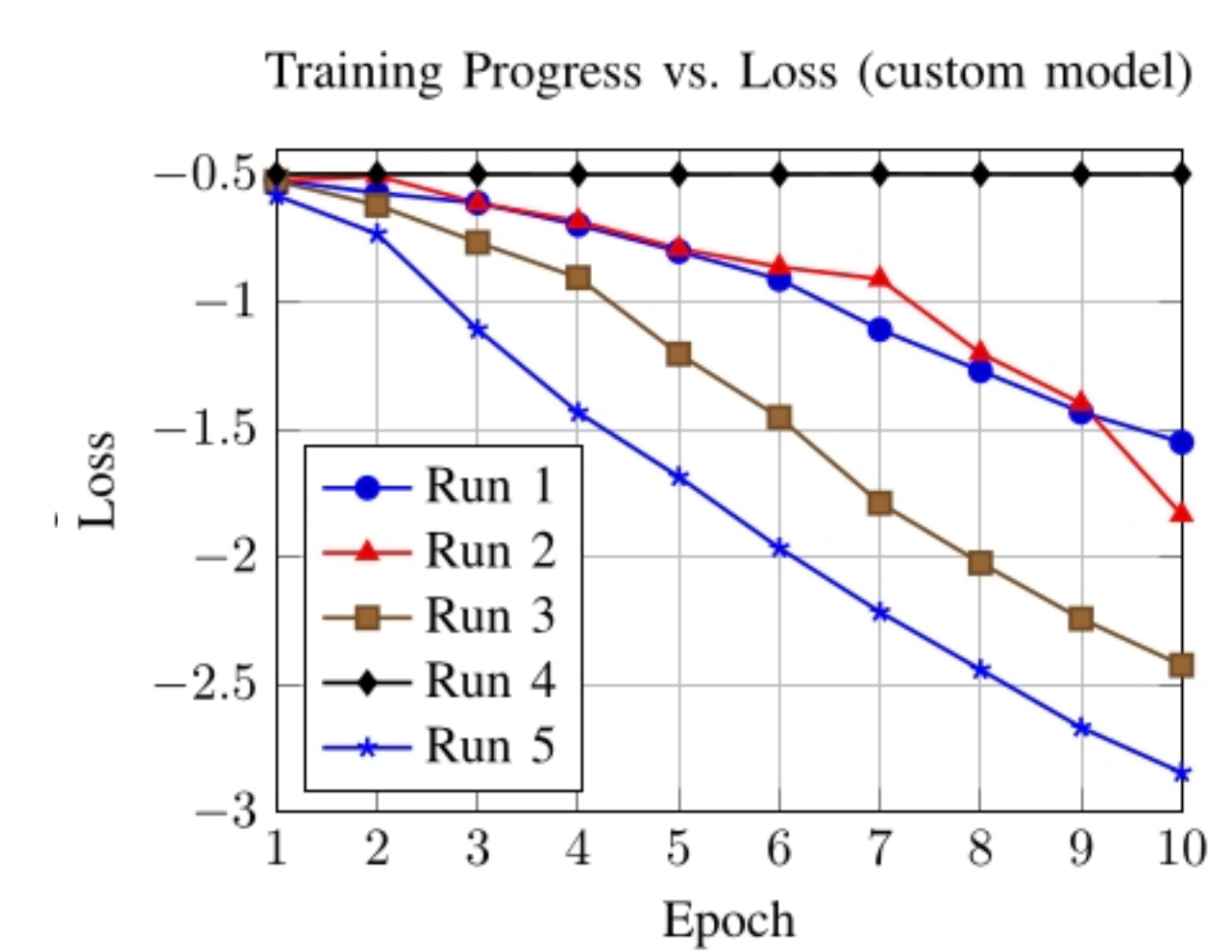
Results of the median loss of the custom HQCNN before and after affing evolutionary-inspired features.

Benchmark with a more challenging dataset.

After performing like the first tutorial on the MNIST written digits dataset, these neural networks were benchmarked on the more challenging fashion MNIST dataset. A subset of classes (0 and 2, "T-shirt/top" and "Pullover") was selected to keep binary classification.



	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-2.6044	-0.4413	-0.8352	-0.4623	-0.7545
accuracy	93%	47%	89%	49%	89%



	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-1.3442	-1.7675	-2.2366	-0.4995	-2.6039
accuracy	91%	94%	93%	50%	93%

Results of 5 tests using the MNIST fashion dataset for binary classification (500 testing samples for 100 training samples, for 10 epochs) for the first tutorial (table 5) and custom HQCNN (table 6).

Conclusion

These results show that the custom HQCNN model performed better than the first tutorial, on the fashion MNIST dataset, achieving high accuracy across different tests. The final loss values indicate that the model converged to a good solution. The training progress also shows a steady decrease in loss over the epochs, indicating that the model was learning effectively. In real IBM hardware tests the accuracy drops due to the natural noise limitations of quantum computers.